To be clear, you can write RSpec after you write your code, just as you can write minitest before you write your code. The design goal of RSpec, however, is to encourage thinking about tests as a way to encourage the design of code yet to be written, and to express those tests in a way that is closer to natural language. Then, as you fill in the code, the specs can continue to act as tests that validate that your code meets your expectations.

Let's start with a simple example of RSpec in action.

Starting to Score Tennis Matches

The scoring system used in lawn tennis originated in the Middle Ages. As players win successive points, their scores are shown as 15, 30, and 40. The next point is a win unless your opponent also has 40. If you're both tied at 40, then different rules apply—the first player with a clear two-point advantage is the winner. Some say the 0, 15, 30, 40 system is a corruption of the fact that scoring used to be done using the quarters of a clock face. Us, we just think those medieval folks enjoyed a good joke.

We want to write a class that handles this scoring system. Let's use RSpec specifications to drive the process. We install RSpec with `gem install rspec`, or place it in our Gemfile, see Chapter 15, Ruby Gems, on page 251. We'll then create our first specification file:

unittesting/bdd/1/ts_spec.rb

```
RSpec.describe "TennisScorer" do
  describe "basic scoring" do
    it "starts with a score of 0-0"
    it "makes the score 15-0 if the server wins a point"
    it "makes the score 0-15 if the receiver wins a point"
    it "makes the score 15-15 after they both win a point"
  end
end
```

This file contains nothing more than a description of the beginning of how a tennis scoring class that we haven't yet written should behave. Inside declaration of the class is a grouping (`describe "basic scoring"`), and inside that is a set of four expectations, all of which start with `it`. We can run this specification using the `rspec` command.

```
$ rspec ts_spec.rb
****
Pending: (Failures listed here are expected and do not affect your suite's status)
  1) TennisScorer basic scoring starts with a score of 0-0
     # Not yet implemented
     # ./ts_spec.rb:3
  2) TennisScorer basic scoring makes the score 15-0 if the server wins a point
     # Not yet implemented
     # ./ts_spec.rb:4
  3) TennisScorer basic scoring makes the score 0-15 if the receiver wins a point
     # Not yet implemented
     # ./ts_spec.rb:5
  4) TennisScorer basic scoring makes the score 15-15 after they both win a point
     # Not yet implemented
     # ./ts_spec.rb:6
Finished in 0.00204 seconds (files took 0.06354 seconds to load)
4 examples, 0 failures, 4 pending
```